

# PY16CHESS SUNFISH

PY-16 FANTASY CONSOLE CARTRIDGE



BY PROF PLANLOZ

2026-05-30

OPEN THIS PDF WITH PY-16 TO PLAY

# PY16CHESS SUNFISH - MANUAL

## DESCRIPTION

PY-16 CHESS (Couch Co-op Edition)

A complete 16-bit chess game for py-16.

Play locally against a friend (with two gamepads)

or challenge the built-in chess AI.

Includes a chess clock, save system, and customizable colors!

PY-16 CHESS (Couch Co-op Edition)

A complete 16-bit chess game for py-16.

Play locally against a friend (with two gamepads)

or challenge the built-in chess AI.

Includes a chess clock, save system, and customizable colors!

## CONTROLS

D-Pad / Arrows : Move cursor  
Z / A-Button : Select piece / Execute move  
Mouse / Left click: Alternative controls (Point & Click)  
S / A (Keyboard) : Save / Load game  
Space / Select : Back to menu / Cancel game

D-Pad / Arrows : Move cursor  
Z / A-Button : Select piece / Execute move  
Mouse / Left click: Alternative controls (Point & Click)  
S / A (Keyboard) : Save / Load game  
Space / Select : Back to menu / Cancel game

## CREDITS

Code, Graphics & UI : Claude, Gemini, ProfPlanloz

AI-Engine : Sunfish (Thomas Ahle)

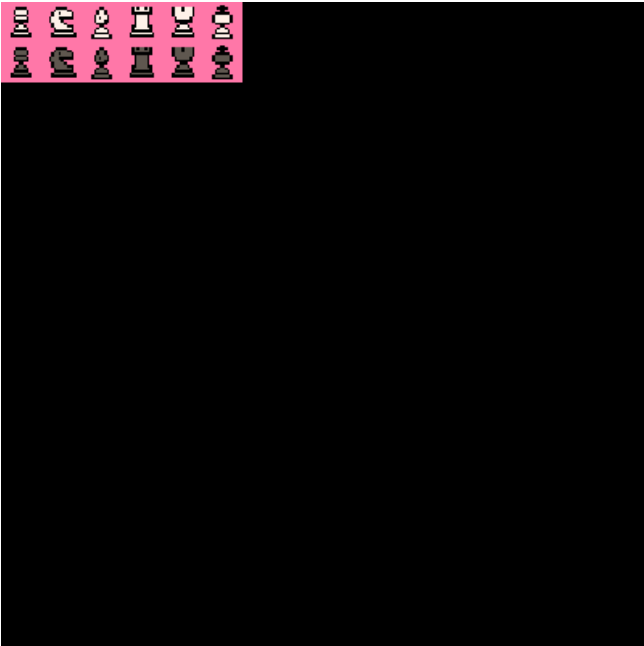
License : GNU General Public License 3(GPLv3)

Code, Graphics & UI : py-16 Gamedev

AI-Engine : Sunfish (Thomas Ahle)

# PY16CHESS SUNFISH - ASSETS

SPRITE SHEET (256x256)



MAP (128x128)



SFX SLOTS

MUSIC TRACKS

# PY16CHESS SUNFISH - CODE LISTING (PAGE 1)

```
1 # @manual
2 # @description
3 # PY-16 CHESS (Couch Co-op Edition)
4 # A complete 16-bit chess game for py-16.
5 # Play locally against a friend (with two gamepads)
6 # or challenge the built-in chess AI.
7 # Includes a chess clock, save system, and customizable colors!
8 #
9 # @controls
10 # D-Pad / Arrows      : Move cursor
11 # Z / A-Button       : Select piece / Execute move
12 # Mouse / Left click: Alternative controls (Point & Click)
13 # S / A (Keyboard)   : Save / Load game
14 # Space / Select     : Back to menu / Cancel game
15 #
16 # @credits
17 # Code, Graphics & UI : Claude, Gemini, ProfPlanloz
18 # AI-Engine           : Sunfish (Thomas Ahle)
19 # License              : GNU General Public License 3(GPLv3)
20 # @end
21
22 import py16
23 import os
24 import json
25
26 import time
27 from itertools import count
28 from collections import namedtuple
29 import sys as _sys
30 _sys.setrecursionlimit(10000)
31
32 # =====
33 # Embedded Sunfish engine - (c) Thomas Dybdahl Ahle, GPLv3.
34 # Source: github.com/thomasahle/sunfish (sunfish.py, 'sunfish 2023').
35 # Only the engine core (piece tables, Position, Searcher) is included; the
36 # UCI front-end is stripped. The cart drives it via the bridge below.
37 # =====
38
39 piece = {"P": 100, "N": 280, "B": 320, "R": 479, "Q": 929, "K": 60000}
40 pst = {
41     'P': ( 0, 0, 0, 0, 0, 0, 0, 0,
42           78, 83, 86, 73, 102, 82, 85, 90,
43           7, 29, 21, 44, 40, 31, 44, 7,
44          -17, 16, -2, 15, 14, 0, 15, -13,
45          -26, 3, 10, 9, 6, 1, 0, -23,
46          -22, 9, 5, -11, -10, -2, 3, -19,
47          -31, 8, -7, -37, -36, -14, 3, -31,
48           0, 0, 0, 0, 0, 0, 0, 0),
49     'N': (-66, -53, -75, -75, -10, -55, -58, -70,
50          -3, -6, 100, -36, 4, 62, -4, -14,
51          10, 67, 1, 74, 73, 27, 62, -2,
52          24, 24, 45, 37, 33, 41, 25, 17,
53          -1, 5, 31, 21, 22, 35, 2, 0,
54          -18, 10, 13, 22, 18, 15, 11, -14,
55          -23, -15, 2, 0, 2, 0, -23, -20,
56          -74, -23, -26, -24, -19, -35, -22, -69),
57     'B': (-59, -78, -82, -76, -23, -107, -37, -50,
58          -11, 20, 35, -42, -39, 31, 2, -22,
59          -9, 39, -32, 41, 52, -10, 28, -14,
60          25, 17, 20, 34, 26, 25, 15, 10,
61          13, 10, 17, 23, 17, 16, 0, 7,
62          14, 25, 24, 15, 8, 25, 20, 15,
63          19, 20, 11, 6, 7, 6, 20, 16,
64          -7, 2, -15, -12, -14, -15, -10, -10),
```

# PY16CHESS SUNFISH - CODE LISTING (PAGE 2)

```

65     'R': ( 35, 29, 33, 4, 37, 33, 56, 50,
66           55, 29, 56, 67, 55, 62, 34, 60,
67           19, 35, 28, 33, 45, 27, 25, 15,
68           0, 5, 16, 13, 18, -4, -9, -6,
69           -28, -35, -16, -21, -13, -29, -46, -30,
70           -42, -28, -42, -25, -25, -35, -26, -46,
71           -53, -38, -31, -26, -29, -43, -44, -53,
72           -30, -24, -18, 5, -2, -18, -31, -32),
73     'Q': ( 6, 1, -8, -104, 69, 24, 88, 26,
74           14, 32, 60, -10, 20, 76, 57, 24,
75           -2, 43, 32, 60, 72, 63, 43, 2,
76           1, -16, 22, 17, 25, 20, -13, -6,
77           -14, -15, -2, -5, -1, -10, -20, -22,
78           -30, -6, -13, -11, -16, -11, -16, -27,
79           -36, -18, 0, -19, -15, -15, -21, -38,
80           -39, -30, -31, -13, -31, -36, -34, -42),
81     'K': ( 4, 54, 47, -99, -99, 60, 83, -62,
82           -32, 10, 55, 56, 56, 55, 10, 3,
83           -62, 12, -57, 44, -67, 28, 37, -31,
84           -55, 50, 11, -4, -19, 13, 0, -49,
85           -55, -43, -52, -28, -51, -47, -8, -50,
86           -47, -42, -43, -79, -64, -32, -29, -32,
87           -4, 3, -14, -50, -57, -18, 13, 4,
88           17, 30, -3, -14, 6, -1, 40, 18),
89 }
90 # Pad tables and join piece and pst dictionaries
91 for k, table in pst.items():
92     padrow = lambda row: (0,) + tuple(x + piece[k] for x in row) + (0,)
93     pst[k] = sum((padrow(table[i * 8 : i * 8 + 8]) for i in range(8)), ())
94     pst[k] = (0,) * 20 + pst[k] + (0,) * 20
95
96 #####
97 # Global constants
98 #####
99
100 # Our board is represented as a 120 character string. The padding allows for
101 # fast detection of moves that don't stay within the board.
102 A1, H1, A8, H8 = 91, 98, 21, 28
103 initial = (
104     "      \n" # 0 - 9
105     "      \n" # 10 - 19
106     " rnbqkbnr\n" # 20 - 29
107     " pppppppp\n" # 30 - 39
108     " ..... \n" # 40 - 49
109     " ..... \n" # 50 - 59
110     " ..... \n" # 60 - 69
111     " ..... \n" # 70 - 79
112     " P P P P P P P P\n" # 80 - 89
113     " R N B Q K B N R\n" # 90 - 99
114     "      \n" # 100 - 109
115     "      \n" # 110 - 119
116 )
117
118 # Lists of possible moves for each piece type.
119 N, E, S, W = -10, 1, 10, -1
120 directions = {
121     "P": (N, N+N, N+W, N+E),
122     "N": (N+N+E, E+N+E, E+S+E, S+S+E, S+S+W, W+S+W, W+N+W, N+N+W),
123     "B": (N+E, S+E, S+W, N+W),
124     "R": (N, E, S, W),
125     "Q": (N, E, S, W, N+E, S+E, S+W, N+W),
126     "K": (N, E, S, W, N+E, S+E, S+W, N+W)
127 }
128

```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 3)

```
129 # Mate value must be greater than 8*queen + 2*(rook+knight+bishop)
130 # King value is set to twice this value such that if the opponent is
131 # 8 queens up, but we got the king, we still exceed MATE_VALUE.
132 # When a MATE is detected, we'll set the score to MATE_UPPER - plies to get there
133 # E.g. Mate in 3 will be MATE_UPPER - 6
134 MATE_LOWER = piece["K"] - 10 * piece["Q"]
135 MATE_UPPER = piece["K"] + 10 * piece["Q"]
136
137 # Constants for tuning search
138 QS = 40
139 QS_A = 140
140 EVAL_ROUGHNESS = 15
141
142
143
144 #####
145 # Chess logic
146 #####
147
148
149 Move = namedtuple("Move", "i j prom")
150
151
152 class Position(namedtuple("Position", "board score wc bc ep kp")):
153     """A state of a chess game
154     board -- a 120 char representation of the board
155     score -- the board evaluation
156     wc -- the castling rights, [west/queen side, east/king side]
157     bc -- the opponent castling rights, [west/king side, east/queen side]
158     ep - the en passant square
159     kp - the king passant square
160     """
161
162     def gen_moves(self):
163         # For each of our pieces, iterate through each possible 'ray' of moves,
164         # as defined in the 'directions' map. The rays are broken e.g. by
165         # captures or immediately in case of pieces such as knights.
166         for i, p in enumerate(self.board):
167             if not p.isupper():
168                 continue
169             for d in directions[p]:
170                 for j in count(i + d, d):
171                     q = self.board[j]
172                     # Stay inside the board, and off friendly pieces
173                     if q.isspace() or q.isupper():
174                         break
175                     # Pawn move, double move and capture
176                     if p == "P":
177                         if d in (N, N + N) and q != ".": break
178                         if d == N + N and (i < A1 + N or self.board[i + N] != "."): break
179                         if (
180                             d in (N + W, N + E)
181                             and q == "."
182                             and j not in (self.ep, self.kp, self.kp - 1, self.kp + 1)
183                             #and j != self.ep and abs(j - self.kp) >= 2
184                         ):
185                             break
186                     # If we move to the last row, we can be anything
187                     if A8 <= j <= H8:
188                         for prom in "NBRQ":
189                             yield Move(i, j, prom)
190                             break
191                     # Move it
192                     yield Move(i, j, "")
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 4)

```
193         # Stop crawlers from sliding, and sliding after captures
194         if p in "PNK" or q.islower():
195             break
196         # Castling, by sliding the rook next to the king
197         if i == A1 and self.board[j + E] == "K" and self.wc[0]:
198             yield Move(j + E, j + W, "")
199         if i == H1 and self.board[j + W] == "K" and self.wc[1]:
200             yield Move(j + W, j + E, "")
201
202     def rotate(self, nullmove=False):
203         """Rotates the board, preserving enpassant, unless nullmove"""
204         return Position(
205             self.board[::-1].swapcase(), -self.score, self.bc, self.wc,
206             119 - self.ep if self.ep and not nullmove else 0,
207             119 - self.kp if self.kp and not nullmove else 0,
208         )
209
210     def move(self, move):
211         i, j, prom = move
212         p, q = self.board[i], self.board[j]
213         put = lambda board, i, p: board[:i] + p + board[i + 1 :]
214         # Copy variables and reset ep and kp
215         board = self.board
216         wc, bc, ep, kp = self.wc, self.bc, 0, 0
217         score = self.score + self.value(move)
218         # Actual move
219         board = put(board, j, board[i])
220         board = put(board, i, ".")
221         # Castling rights, we move the rook or capture the opponent's
222         if i == A1: wc = (False, wc[1])
223         if i == H1: wc = (wc[0], False)
224         if j == A8: bc = (bc[0], False)
225         if j == H8: bc = (False, bc[1])
226         # Castling
227         if p == "K":
228             wc = (False, False)
229             if abs(j - i) == 2:
230                 kp = (i + j) // 2
231                 board = put(board, A1 if j < i else H1, ".")
232                 board = put(board, kp, "R")
233         # Pawn promotion, double move and en passant capture
234         if p == "P":
235             if A8 <= j <= H8:
236                 board = put(board, j, prom)
237             if j - i == 2 * N:
238                 ep = i + N
239             if j == self.ep:
240                 board = put(board, j + S, ".")
241         # We rotate the returned position, so it's ready for the next player
242         return Position(board, score, wc, bc, ep, kp).rotate()
243
244     def value(self, move):
245         i, j, prom = move
246         p, q = self.board[i], self.board[j]
247         # Actual move
248         score = pst[p][j] - pst[p][i]
249         # Capture
250         if q.islower():
251             score += pst[q.upper()][119 - j]
252         # Castling check detection
253         if abs(j - self.kp) < 2:
254             score += pst["K"][119 - j]
255         # Castling
256         if p == "K" and abs(i - j) == 2:
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 5)

```
257         score += pst["R"][(i + j) // 2]
258         score -= pst["R"][A1 if j < i else H1]
259     # Special pawn stuff
260     if p == "P":
261         if A8 <= j <= H8:
262             score += pst[prom][j] - pst["P"][j]
263         if j == self.ep:
264             score += pst["P"][119 - (j + S)]
265     return score
266
267
268 #####
269 # Search logic
270 #####
271
272 # lower <= s(pos) <= upper
273 Entry = namedtuple("Entry", "lower upper")
274
275
276 class Searcher:
277     def __init__(self):
278         self.tp_score = {}
279         self.tp_move = {}
280         self.history = set()
281         self.nodes = 0
282
283     def bound(self, pos, gamma, depth, can_null=True):
284         """ Let s* be the "true" score of the sub-tree we are searching.
285             The method returns r, where
286             if gamma > s* then s* <= r < gamma (A better upper bound)
287             if gamma <= s* then gamma <= r <= s* (A better lower bound) """
288         self.nodes += 1
289
290         # Depth <= 0 is QSearch. Here any position is searched as deeply as is needed for
291         # calmness, and from this point on there is no difference in behaviour depending o
292         # depth, so so there is no reason to keep different depths in the transposition ta
293         depth = max(depth, 0)
294
295         # Sunfish is a king-capture engine, so we should always check if we
296         # still have a king. Notice since this is the only termination check,
297         # the remaining code has to be comfortable with being mated, stalemated
298         # or able to capture the opponent king.
299         if pos.score <= -MATE_LOWER:
300             return -MATE_UPPER
301
302         # Look in the table if we have already searched this position before.
303         # We also need to be sure, that the stored search was over the same
304         # nodes as the current search.
305         entry = self.tp_score.get((pos, depth, can_null), Entry(-MATE_UPPER, MATE_UPPER))
306         if entry.lower >= gamma: return entry.lower
307         if entry.upper < gamma: return entry.upper
308
309         # Let's not repeat positions. We don't chat
310         # - at the root (can_null=False) since it is in history, but not a draw.
311         # - at depth=0, since it would be expensive and break "futility pruning".
312         if can_null and depth > 0 and pos in self.history:
313             return 0
314
315         # Generator of moves to search in order.
316         # This allows us to define the moves, but only calculate them if needed.
317         def moves():
318             # First try not moving at all. We only do this if there is at least one major
319             # piece left on the board, since otherwise zugzwangs are too dangerous.
320             # FIXME: We also can't null move if we can capture the opponent king.
```



## PY16CHESS SUNFISH - CODE LISTING (PAGE 6)

```
321         # Since if we do, we won't spot illegal moves that could lead to stalemate.
322         # For now we just solve this by not using null-move in very unbalanced positio
323         # TODO: We could actually use null-move in QS as well. Not sure it would be ve
324         # But still.... We just have to move stand-pat to be before null-move.
325         #if depth > 2 and can_null and any(c in pos.board for c in "RBNQ"):
326         #if depth > 2 and can_null and any(c in pos.board for c in "RBNQ") and abs(pos
327         if depth > 2 and can_null and abs(pos.score) < 500:
328             yield None, -self.bound(pos.rotate(nullmove=True), 1 - gamma, depth - 3)
329
330         # For QSearch we have a different kind of null-move, namely we can just stop
331         # and not capture anything else.
332         if depth == 0:
333             yield None, pos.score
334
335         # Look for the strongest ove from last time, the hash-move.
336         killer = self.tp_move.get(pos)
337
338         # If there isn't one, try to find one with a more shallow search.
339         # This is known as Internal Iterative Deepening (IID). We set
340         # can_null=True, since we want to make sure we actually find a move.
341         if not killer and depth > 2:
342             self.bound(pos, gamma, depth - 3, can_null=False)
343             killer = self.tp_move.get(pos)
344
345         # If depth == 0 we only try moves with high intrinsic score (captures and
346         # promotions). Otherwise we do all moves. This is called quiescent search.
347         val_lower = QS - depth * QS_A
348
349         # Only play the move if it would be included at the current val-limit,
350         # since otherwise we'd get search instability.
351         # We will search it again in the main loop below, but the tp will fix
352         # things for us.
353         if killer and pos.value(killer) >= val_lower:
354             yield killer, -self.bound(pos.move(killer), 1 - gamma, depth - 1)
355
356         # Then all the other moves
357         for val, move in sorted(((pos.value(m), m) for m in pos.gen_moves()), reverse=
358             # Quiescent search
359             if val < val_lower:
360                 break
361
362             # If the new score is less than gamma, the opponent will for sure just
363             # stand pat, since ""pos.score + val < gamma ==> -(pos.score + val) >= 1-g
364             # This is known as futility pruning.
365             if depth <= 1 and pos.score + val < gamma:
366                 # Need special case for MATE, since it would normally be caught
367                 # before standing pat.
368                 yield move, pos.score + val if val < MATE_LOWER else MATE_UPPER
369                 # We can also break, since we have ordered the moves by value,
370                 # so it can't get any better than this.
371                 break
372
373             yield move, -self.bound(pos.move(move), 1 - gamma, depth - 1)
374
375         # Run through the moves, shortcutting when possible
376         best = -MATE_UPPER
377         for move, score in moves():
378             best = max(best, score)
379             if best >= gamma:
380                 # Save the move for pv construction and killer heuristic
381                 if move is not None:
382                     self.tp_move[pos] = move
383                 break
384
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 7)

```
385     # Stalemate checking is a bit tricky: Say we failed low, because
386     # we can't (legally) move and so the (real) score is -infty.
387     # At the next depth we are allowed to just return r, -infty <= r < gamma,
388     # which is normally fine.
389     # However, what if gamma = -10 and we don't have any legal moves?
390     # Then the score is actually a draw and we should fail high!
391     # Thus, if best < gamma and best < 0 we need to double check what we are doing.
392
393     # We will fix this problem another way: We add the requirement to bound, that
394     # it always returns MATE_UPPER if the king is capturable. Even if another move
395     # was also sufficient to go above gamma. If we see this value we know we are either
396     # mate, or stalemate. It then suffices to check whether we're in check.
397
398     # Note that at low depths, this may not actually be true, since maybe we just pruned
399     # all the legal moves. So sunfish may report "mate", but then after more search
400     # realize it's not a mate after all. That's fair.
401
402     # This is too expensive to test at depth == 0
403     if depth > 2 and best == -MATE_UPPER:
404         flipped = pos.rotate(nullmove=True)
405         # Hopefully this is already in the TT because of null-move
406         in_check = self.bound(flipped, MATE_UPPER, 0) == MATE_UPPER
407         best = -MATE_LOWER if in_check else 0
408
409     # Table part 2
410     if best >= gamma:
411         self.tp_score[pos, depth, can_null] = Entry(best, entry.upper)
412     if best < gamma:
413         self.tp_score[pos, depth, can_null] = Entry(entry.lower, best)
414
415     return best
416
417 def search(self, history):
418     """Iterative deepening MTD-bi search"""
419     self.nodes = 0
420     self.history = set(history)
421     self.tp_score.clear()
422
423     gamma = 0
424     # In finished games, we could potentially go far enough to cause a recursion
425     # limit exception. Hence we bound the ply. We also can't start at 0, since
426     # that's quiescent search, and we don't always play legal moves there.
427     for depth in range(1, 1000):
428         # The inner loop is a binary search on the score of the position.
429         # Inv: lower <= score <= upper
430         # 'while lower != upper' would work, but it's too much effort to spend
431         # on what's probably not going to change the move played.
432         lower, upper = -MATE_LOWER, MATE_LOWER
433         while lower < upper - EVAL_ROUGHNESS:
434             score = self.bound(history[-1], gamma, depth, can_null=False)
435             if score >= gamma:
436                 lower = score
437             if score < gamma:
438                 upper = score
439             yield depth, gamma, score, self.tp_move.get(history[-1])
440             gamma = (lower + upper + 1) // 2
441
442 # ===== end of embedded Sunfish engine =====
443
444 # @manual
445 # @description
446 # PY-16 CHESS (Couch Co-op Edition)
447 # A complete 16-bit chess game for py-16.
448 # Play locally against a friend (with two gamepads)
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 8)

```
449 # or challenge the built-in chess AI.
450 # Includes a chess clock, save system, and customizable colors!
451 #
452 # @controls
453 # D-Pad / Arrows      : Move cursor
454 # Z / A-Button       : Select piece / Execute move
455 # Mouse / Left click: Alternative controls (Point & Click)
456 # S / A (Keyboard)   : Save / Load game
457 # Space / Select     : Back to menu / Cancel game
458 #
459 # @credits
460 # Code, Graphics & UI : py-16 Gamedev
461 # AI-Engine           : Sunfish (Thomas Ahle)
462 # @end
463
464 # --- GLOBAL STATE VARIABLES ---
465 board = []
466 turn = 'w'
467 selected_sq = None
468 valid_moves = []
469 game_status = 'play' # 'play', 'menu', 'settings', 'ai_thinking', 'promote', 'checkmate',
470 winner = None
471 vs_ai = False        # Are we playing against the Sunfish AI?
472
473 # Variables for advanced chess rules
474 king_moved = {'w': False, 'b': False}
475 rook_moved = {'w': [False, False], 'b': [False, False]}
476 en_passant_target = None
477 promotion_state = None
478 was_capture_saved = False
479
480 # Undo: stack of full state snapshots, one per played half-move
481 undo_stack = []
482 MAX_UNDO = 40          # cap memory; plenty for casual play
483 pending_move = None    # (sr,sc,r,c) awaiting confirmation when confirm_moves is on
484
485 # 50-move rule, threefold repetition and draw bookkeeping
486 halfmove_clock = 0     # half-moves since last pawn move or capture
487 position_history = []  # FEN-like keys, for threefold repetition
488 draw_reason = ""       # text shown when a game ends in a draw
489
490 # UI & Settings Variables
491 show_msg = ""
492 msg_timer = 0
493 sound_on = True
494 difficulty = 1          # 0: Easy, 1: Medium, 2: Hard
495 confirm_moves = False  # if True, a selected move must be confirmed before it plays
496
497 # Time Control Variables
498 time_options = [0, 60, 300, 600, 900, 1200, 1800, 3600]
499 time_option_strs = ["NONE", "1:00", "5:00", "10:00", "15:00", "20:00", "30:00", "60:00"]
500 time_idx = 0
501 frames_white = 0
502 frames_black = 0
503
504 # Background Variables
505 bg_options = [13, 1, 2, 3, 5, 12, 0, 4, 8, 11, 14, 15]
506 bg_names = ["INDIGO", "DARK BLUE", "PURPLE", "GREEN", "GRAY", "BLUE", "BLACK", "BROWN", "R
507 bg_idx = 0
508
509 # Controller Cursor Variables
510 cursor_r = 4
511 cursor_c = 4
512 promote_cursor = 0
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 9)

```
513 menu_cursor = 0
514 settings_cursor = 0
515 demo_timer = 40 # Timer for the background AI in the menu
516
517 # Layout Constants
518 BOARD_X = 64
519 BOARD_Y = 48
520 SQ_SIZE = 16
521
522 # --- COUCH CO-OP HELPER ---
523 def get_active_controller():
524     """Input source to query for the side whose turn it is.
525
526     Returns 0 ('any source' = keyboard + Player-1 gamepad) for White and for
527     pass-and-play, exactly like the menu does, so the keyboard always works
528     whether or not a gamepad is plugged in. Only a real second controller in a
529     local match gets its own slot 2.
530     """
531     # Local 2-player with a real second controller: Black is Player 2.
532     if turn == 'b' and not vs_ai and py16.player_connected(2):
533         return 2
534     # Everyone else (White, pass-and-play Black, vs-AI): any source.
535     return 0
536
537
538 def get_active_player_label():
539     """Player number (1 or 2) to show in the on-screen status line."""
540     return 2 if get_active_controller() == 2 else 1
541
542 # --- SOUND HELPER ---
543 def play_sound(*args, **kwargs):
544     """Plays a tone, but only if sound is activated."""
545     if sound_on:
546         py16.tone(*args, **kwargs)
547
548 # --- SUNFISH AI: EVALUATION TABLES (PST) ---
549 SUNFISH_PST = {
550     1: [ 0, 0, 0, 0, 0, 0, 0, 0,
551         78, 83, 86, 73, 102, 82, 85, 90,
552         7, 29, 21, 44, 40, 31, 44, 7,
553        -17, 16, -2, 15, 14, 0, 15, -13,
554        -26, 3, 10, 9, 6, 1, 0, -23,
555        -22, 9, 5, -11, -10, -2, 3, -19,
556        -31, 8, -7, -37, -36, -14, 3, -31,
557         0, 0, 0, 0, 0, 0, 0, 0],
558     2: [-66, -53, -75, -75, -10, -55, -58, -70,
559        -3, -6, 100, -36, 4, 62, -4, -14,
560        10, 67, 1, 74, 73, 27, 62, -2,
561        24, 24, 45, 37, 33, 41, 25, 17,
562        -1, 5, 31, 21, 22, 35, 2, 0,
563        -18, 10, 13, 22, 18, 15, 11, -14,
564        -23, -15, 2, 0, 2, 0, -23, -20,
565        -74, -23, -26, -24, -19, -35, -22, -69],
566     3: [-59, -78, -82, -76, -23, -107, -37, -50,
567        -11, 20, 35, -42, -39, 31, 2, -22,
568        -9, 39, -32, 41, 52, -10, 28, -14,
569        25, 17, 20, 34, 26, 25, 15, 10,
570        13, 10, 17, 23, 17, 16, 0, 7,
571        14, 25, 24, 15, 8, 25, 20, 15,
572        19, 20, 11, 6, 7, 6, 20, 16,
573        -7, 2, -15, -12, -14, -15, -10, -10],
574     4: [ 35, 29, 33, 4, 37, 33, 56, 50,
575        55, 29, 56, 67, 55, 62, 34, 60,
576        19, 35, 28, 33, 45, 27, 25, 15,
```

# PY16CHESS SUNFISH - CODE LISTING (PAGE 10)

```

577         0, 5, 16, 13, 18, -4, -9, -6,
578         -28, -35, -16, -21, -13, -29, -46, -30,
579         -42, -28, -42, -25, -25, -35, -26, -46,
580         -53, -38, -31, -26, -29, -43, -44, -53,
581         -30, -24, -18, 5, -2, -18, -31, -32],
582     5: [ 6, 1, -8, -104, 69, 24, 88, 26,
583         14, 32, 60, -10, 20, 76, 57, 24,
584         -2, 43, 32, 60, 72, 63, 43, 2,
585         1, -16, 22, 17, 25, 20, -13, -6,
586         -14, -15, -2, -5, -1, -10, -20, -22,
587         -30, -6, -13, -11, -16, -11, -16, -27,
588         -36, -18, 0, -19, -15, -15, -21, -38,
589         -39, -30, -31, -13, -31, -36, -34, -42],
590     6: [ 4, 54, 47, -99, -99, 60, 83, -62,
591         -32, 10, 55, 56, 56, 55, 10, 3,
592         -62, 12, -57, 44, -67, 28, 37, -31,
593         -55, 50, 11, -4, -19, 13, 0, -49,
594         -55, -43, -52, -28, -51, -47, -8, -50,
595         -47, -42, -43, -79, -64, -32, -29, -32,
596         -4, 3, -14, -50, -57, -18, 13, 4,
597         17, 30, -3, -14, 6, -1, 40, 18]
598 }
599 PIECE_VALUES = {1: 100, 2: 280, 3: 320, 4: 479, 5: 929, 6: 60000}
600
601 # --- PIXEL ART DEFINITIONS (16x16) ---
602 PIXEL_ART = {
603     1: [ "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          ",
604         "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          ",
605         "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          ",
606         "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          ",
607         "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          ",
608         "          ", "          ", "          ", "          ", "          ", "          ", "          ", "          "],
609 }
610
611 def generate_sprites():
612     """Draws the pixel art dynamically into the py-16 spritesheet."""
613     py16.palt(14, True) # 14 (Pink) becomes the transparent mask
614     py16.palt(0, False) # Black remains explicitly visible
615
616     for ptype, art in PIXEL_ART.items():
617         base_x = (ptype - 1) * 16
618         for row_idx, row_str in enumerate(art):
619             for col_idx, char in enumerate(row_str):
620                 cx = base_x + col_idx
621                 cy_w = row_idx # Rows 0-15 for White
622                 cy_b = 16 + row_idx # Rows 16-31 for Black
623
624                 c_w, c_b = 14, 14
625                 if char == '#':
626                     c_w, c_b = 0, 0
627                 elif char == '.':
628                     c_w, c_b = 7, 5
629
630                 py16.sset(cx, cy_w, c_w)
631                 py16.sset(cx, cy_b, c_b)
632
633 def reset_board(vs_sunfish=False):
634     """Resets the board for a new game."""
635     global board, turn, selected_sq, valid_moves, winner, vs_ai
636     global king_moved, rook_moved, en_passant_target, promotion_state, was_capture_saved
637     global cursor_r, cursor_c, promote_cursor, show_msg, msg_timer
638     global frames_white, frames_black
639     global halfmove_clock, position_history, draw_reason, undo_stack
640

```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 11)

```
641     vs_ai = vs_sunfish
642     board = [
643         [-4, -2, -3, -5, -6, -3, -2, -4],
644         [-1, -1, -1, -1, -1, -1, -1, -1],
645         [ 0,  0,  0,  0,  0,  0,  0,  0],
646         [ 0,  0,  0,  0,  0,  0,  0,  0],
647         [ 0,  0,  0,  0,  0,  0,  0,  0],
648         [ 0,  0,  0,  0,  0,  0,  0,  0],
649         [ 1,  1,  1,  1,  1,  1,  1,  1],
650         [ 4,  2,  3,  5,  6,  3,  2,  4]
651     ]
652     turn = 'w'
653     selected_sq = None
654     valid_moves = []
655     winner = None
656
657     king_moved = {'w': False, 'b': False}
658     rook_moved = {'w': [False, False], 'b': [False, False]}
659     en_passant_target = None
660     promotion_state = None
661     was_capture_saved = False
662     cursor_r = 4
663     cursor_c = 4
664     promote_cursor = 0
665     show_msg = ""
666     msg_timer = 0
667     frames_white = 0
668     frames_black = 0
669     halfmove_clock = 0
670     position_history = []
671     draw_reason = ""
672     undo_stack = []
673     _ai_reset_search()      # drop any in-progress AI search from a previous game
674     record_position()
675
676 def init():
677     """Starting point of the cartridge."""
678     global game_status
679     reset_board()
680     game_status = 'menu'
681     generate_sprites()
682
683 # --- SAVE / LOAD ---
684 # One single list of persisted fields, so save() and load() can never drift
685 # out of sync (the old code maintained two hand-written copies).
686 SAVE_REQUIRED = ["board", "turn", "king_moved", "rook_moved",
687                 "en_passant_target", "game_status", "winner"]
688 SAVE_OPTIONAL = {
689     "vs_ai": False, "sound_on": True, "difficulty": 1, "time_idx": 0,
690     "frames_white": 0, "frames_black": 0, "bg_idx": 0,
691     "halfmove_clock": 0, "position_history": [], "draw_reason": "",
692     "confirm_moves": False,
693 }
694 SAVE_FIELDS = SAVE_REQUIRED + list(SAVE_OPTIONAL)
695
696
697 def save_dir():
698     """Savegame directory. Lives under ~/.py16/ like the rest of py-16,
699     and can be overridden with the PY16_CHESS_SAVE_DIR environment variable."""
700     env = os.environ.get("PY16_CHESS_SAVE_DIR")
701     if env:
702         return os.path.expanduser(env)
703     return os.path.expanduser(os.path.join("~", ".py16", "chess_saves"))
704
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 12)

```
705
706 def _load_candidates():
707     """New location first, then the legacy ./chess_games/ folder for migration."""
708     return [
709         os.path.join(save_dir(), "savegame.json"),
710         os.path.join("chess_games", "savegame.json"),
711     ]
712
713
714 def _validate_state(s):
715     """Raises ValueError if the parsed save is not a usable game state."""
716     for k in SAVE_REQUIRED:
717         if k not in s:
718             raise ValueError("missing field: " + k)
719     b = s["board"]
720     if not isinstance(b, list) or len(b) != 8:
721         raise ValueError("board must have 8 rows")
722     for row in b:
723         if not isinstance(row, list) or len(row) != 8:
724             raise ValueError("board rows must have 8 columns")
725         for v in row:
726             if not isinstance(v, int) or not (-6 <= v <= 6):
727                 raise ValueError("illegal board value")
728     if s["turn"] not in ("w", "b"):
729         raise ValueError("turn must be 'w' or 'b'")
730
731
732 def _flash(msg, freq, wave):
733     global show_msg, msg_timer
734     show_msg = msg
735     msg_timer = 90
736     play_sound(freq, 100, wave)
737
738
739 def save_game():
740     try:
741         d = save_dir()
742         os.makedirs(d, exist_ok=True)
743         state = {k: globals()[k] for k in SAVE_FIELDS}
744         path = os.path.join(d, "savegame.json")
745         tmp = path + ".tmp"
746         with open(tmp, "w") as f:
747             json.dump(state, f)
748         os.replace(tmp, path) # atomic: never leaves a half-written file
749         _flash("SAVED!", 600, py16.WAVE_SQUARE)
750     except Exception:
751         _flash("SAVE ERROR!", 200, py16.WAVE_SAW)
752
753
754 def load_game():
755     global selected_sq, valid_moves, promotion_state, cursor_r, cursor_c
756
757     state = None
758     for path in _load_candidates():
759         if os.path.exists(path):
760             try:
761                 with open(path, "r") as f:
762                     state = json.load(f)
763                     _validate_state(state)
764                     break
765             except Exception:
766                 _flash("SAVE CORRUPT!", 200, py16.WAVE_SAW)
767                 return
768     if state is None:
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 13)

```
769         _flash("NO SAVEGAME!", 200, py16.WAVE_SAW)
770     return
771
772     # Commit only after validation succeeded, so a corrupt file can never leave
773     # the game in a half-loaded state.
774     for k in SAVE_REQUIRED:
775         globals()[k] = state[k]
776     for k, default in SAVE_OPTIONAL.items():
777         globals()[k] = state.get(k, default)
778
779     ep = globals()["en_passant_target"]
780     globals()["en_passant_target"] = tuple(ep) if ep else None
781     selected_sq = None
782     valid_moves = []
783     promotion_state = None
784     cursor_r, cursor_c = 4, 4
785
786     _flash("GAME LOADED!", 800, py16.WAVE_SAW)
787
788
789 # --- AI SYSTEM (SUNFISH EVALUATION) ---
790 def evaluate_board(bld):
791     """Evaluates the board based on Sunfish material and position tables."""
792     score = 0
793     for r in range(8):
794         for c in range(8):
795             p = bld[r][c]
796             if p == 0: continue
797             ptype = abs(p)
798             is_white = (p > 0)
799
800             val = PIECE_VALUES[ptype]
801
802             # Calculate Sunfish PST index
803             if is_white:
804                 idx = r * 8 + c
805             else:
806                 idx = (7 - r) * 8 + c # For Black the evaluation is mirrored
807
808             pst_val = SUNFISH_PST[ptype][idx]
809
810             if is_white:
811                 score += val + pst_val
812             else:
813                 score -= (val + pst_val)
814     return score
815
816 def evaluate_move(sr, sc, tr, tc, active_color):
817     """Simulates a move on a local board copy and returns the opponent's best
818     immediate capture reply (1-Ply + capture-only quiescence). The real game
819     board is never touched, which removes a whole class of make/unmake bugs."""
820     global en_passant_target
821     bld = [row[:] for row in board]
822     p = bld[sr][sc]
823     bld[tr][tc] = p
824     bld[sr][sc] = 0
825
826     # get_pseudo_moves() reads the en-passant target from the module global,
827     # so align it with the simulated position for the duration of this call.
828     saved_ep = en_passant_target
829     if abs(p) == 1 and abs(sr - tr) == 2:
830         en_passant_target = ((sr + tr) // 2, sc)
831     else:
832         en_passant_target = None
```



## PY16CHESS SUNFISH - CODE LISTING (PAGE 14)

```
833
834     worst_score = float('inf') if active_color == 'w' else -float('inf')
835     has_captures = False
836     enemy_sign = -1 if active_color == 'w' else 1
837
838     # Strongest opponent capture in the resulting position.
839     for wr in range(8):
840         for wc in range(8):
841             if (bld[wr][wc] * enemy_sign) > 0:
842                 for wtr, wtc in get_pseudo_moves(wr, wc, bld):
843                     target = bld[wtr][wtc]
844                     if (target * enemy_sign) < 0 and abs(target) != 6:
845                         has_captures = True
846                         bld[wtr][wtc] = bld[wr][wc]
847                         bld[wr][wc] = 0
848                         score = evaluate_board(bld)
849                         if active_color == 'w':
850                             worst_score = min(worst_score, score)
851                         else:
852                             worst_score = max(worst_score, score)
853                         bld[wr][wc] = bld[wtr][wtc]
854                         bld[wtr][wtc] = target
855
856     if not has_captures:
857         worst_score = evaluate_board(bld)
858
859     en_passant_target = saved_ep
860     return worst_score
861
862
863 # --- SUNFISH BRIDGE -----
864 # Difficulty -> (total time budget in seconds, max search depth).
865 # The search is frame-stepped (see step_sunfish_search), but a single
866 # iterative-deepening step is atomic - Sunfish's generator yields only at the
867 # MTD-bi steps within a depth, so a deep step can briefly stall a frame. That
868 # occasional hitch is accepted in exchange for genuinely strong play.
869 SUNFISH_LEVELS = {0: (0.05, 2), 1: (0.50, 6), 2: (2.50, 24)}
870 # Max seconds of engine work per frame, so the search can be spread across
871 # frames instead of freezing the 60 FPS loop in one big block.
872 AI_FRAME_SLICE = 0.045
873 # Sunfish promotion letter -> cart piece type.
874 _PROM_TO_CART = {'N': 2, 'B': 3, 'R': 4, 'Q': 5, '': 5}
875
876 # In-progress (frame-stepped) search state.
877 _ai_search_gen = None
878 _ai_best_move = None
879 _ai_start_time = 0.0
880 _ai_pos_turn = None
881 _ai_reached_depth = 0
882
883
884 def _cart_to_sunfish_board(cart_board):
885     """Builds Sunfish's 120-char padded board string (White = uppercase)."""
886     rows = ["\n", "\n"]
887     for r in range(8):
888         line = ""
889         for c in range(8):
890             v = cart_board[r][c]
891             if v == 0:
892                 line += "."
893             else:
894                 ch = "PNBRQK"[abs(v) - 1]
895                 line += ch if v > 0 else ch.lower()
896     rows.append(line + "\n")
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 15)

```
897     rows.append("          \n")
898     rows.append("          \n")
899     return "".join(rows)
900
901
902 def _sunfish_static_score(board_str):
903     """White-perspective sum of piece-square values (matches Sunfish)."""
904     s = 0
905     for i, p in enumerate(board_str):
906         if p.isupper():
907             s += pst[p][i]
908         elif p.islower():
909             s -= pst[p.upper()][119 - i]
910     return s
911
912
913 def _cart_to_sunfish_position():
914     """Translates the live cart state into a Sunfish Position ready to search."""
915     bstr = _cart_to_sunfish_board(board)
916     score = _sunfish_static_score(bstr)
917     wq = (not king_moved['w']) and (not rook_moved['w'][0]) and board[7][0] == 4
918     wk = (not king_moved['w']) and (not rook_moved['w'][1]) and board[7][7] == 4
919     bq = (not king_moved['b']) and (not rook_moved['b'][0]) and board[0][0] == -4
920     bk = (not king_moved['b']) and (not rook_moved['b'][1]) and board[0][7] == -4
921     wc = (bool(wq), bool(wk))      # (queenside/A1, kingside/H1)
922     bc = (bool(bk), bool(bq))      # (kingside/H8, queenside/A8) per Position.move
923     ep = 0
924     if en_passant_target:
925         er, ec = en_passant_target
926         ep = 21 + er * 10 + ec
927     pos = Position(bstr, score, wc, bc, ep, 0)
928     if turn == 'b':
929         pos = pos.rotate()          # make the side to move 'White' internally
930     return pos
931
932
933 def _sunfish_move_to_cart(mv, color):
934     """Sunfish Move -> (sr, sc, tr, tc, prom_char), or None if off the board."""
935     i, j = mv.i, mv.j
936     if color == 'b':                # undo the rotation applied for black
937         i, j = 119 - i, 119 - j
938     sr, sc = divmod(i - 21, 10)
939     tr, tc = divmod(j - 21, 10)
940     if not (0 <= sr < 8 and 0 <= sc < 8 and 0 <= tr < 8 and 0 <= tc < 8):
941         return None
942     return (sr, sc, tr, tc, mv.prom)
943
944
945 def _sunfish_best_move():
946     """Synchronous full search (used by tests). Returns (sr, sc, tr, tc) or None."""
947     pos = _cart_to_sunfish_position()
948     budget, max_depth = SUNFISH_LEVELS.get(difficulty, (0.40, 5))
949     searcher = Searcher()
950     start = time.time()
951     best = None
952     for depth, gamma, score, mv in searcher.search([pos]):
953         if score >= gamma and mv is not None:
954             best = mv
955         if best is not None and (depth >= max_depth or time.time() - start > budget):
956             break
957         if depth >= 40:
958             break
959     if best is None:
960         return None
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 16)

```
961     conv = _sunfish_move_to_cart(best, turn)
962     if conv is None:
963         return None
964     sr, sc, tr, tc, _ = conv
965     return (sr, sc, tr, tc)
966
967
968 def _play_random_move(color):
969     """Plays a random legal move (used to make 'Easy' blunder). True if it moved."""
970     import random
971     sign = 1 if color == 'w' else -1
972     moves = []
973     for r in range(8):
974         for c in range(8):
975             if board[r][c] * sign > 0:
976                 for tr, tc in get_legal_moves(r, c):
977                     moves.append((r, c, tr, tc))
978     if not moves:
979         return False
980     sr, sc, tr, tc = random.choice(moves)
981     execute_move(sr, sc, tr, tc)
982     return True
983
984
985 def _ai_reset_search():
986     global _ai_search_gen, _ai_best_move, _ai_start_time, _ai_pos_turn, _ai_reached_depth
987     _ai_search_gen = None
988     _ai_best_move = None
989     _ai_start_time = 0.0
990     _ai_pos_turn = None
991     _ai_reached_depth = 0
992
993
994 def _finish_ai_with_fallback():
995     """If Sunfish errors out, fall back to the simple built-in evaluator."""
996     _ai_reset_search()
997     do_ai_turn('b')                # also resets 'ai_thinking' -> 'play'
998
999
1000 def _apply_ai_best_move():
1001     """Convert the chosen Sunfish move, validate it, and play it on the cart."""
1002     global game_status
1003     mv = _ai_best_move
1004     color = _ai_pos_turn
1005     _ai_reset_search()
1006     if mv is not None:
1007         conv = _sunfish_move_to_cart(mv, color)
1008         if conv is not None:
1009             sr, sc, tr, tc, prom = conv
1010             if get_piece_color(board[sr][sc]) == 'b' and (tr, tc) in get_legal_moves(sr, s
1011                 execute_move(sr, sc, tr, tc, ai_promo=_PROM_TO_CART.get(prom, 5))
1012                 if game_status == 'ai_thinking':
1013                     game_status = 'play'
1014             return
1015     do_ai_turn('b')                # fallback
1016
1017
1018 def step_sunfish_search():
1019     """Advance the AI search by ~one frame of work; play the move once the
1020     time budget or depth target is reached. Keeps the UI responsive."""
1021     global _ai_search_gen, _ai_best_move, _ai_start_time, _ai_pos_turn, game_status, _ai_r
1022
1023     # First frame of a new search: set it up.
1024     if _ai_search_gen is None:
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 17)

```
1025     # Easy occasionally blunders like a beginner.
1026     if difficulty == 0 and py16.rnd(100) < 35:
1027         if _play_random_move('b'):
1028             if game_status == 'ai_thinking':
1029                 game_status = 'play'
1030                 _ai_reset_search()
1031             return
1032     try:
1033         pos = _cart_to_sunfish_position()
1034         _ai_pos_turn = turn
1035         _ai_search_gen = Searcher().search([pos])
1036         _ai_start_time = time.time()
1037         _ai_best_move = None
1038     except Exception:
1039         _finish_ai_with_fallback()
1040     return
1041
1042     budget, max_depth = SUNFISH_LEVELS.get(difficulty, (0.30, 4))
1043     frame_start = time.time()
1044     done = False
1045     try:
1046         while time.time() - frame_start < AI_FRAME_SLICE:
1047             # Stop before launching another (deeper, more expensive) step once
1048             # we already have a move and have hit the depth target or time budget.
1049             if _ai_best_move is not None and (
1050                 _ai_reached_depth >= max_depth
1051                 or time.time() - _ai_start_time > budget):
1052                 done = True
1053                 break
1054             depth, gamma, score, mv = next(_ai_search_gen)
1055             _ai_reached_depth = depth
1056             if score >= gamma and mv is not None:
1057                 _ai_best_move = mv
1058             if depth >= 40:
1059                 done = True
1060                 break
1061     except StopIteration:
1062         done = True
1063     except Exception:
1064         _finish_ai_with_fallback()
1065     return
1066
1067     if done:
1068         _apply_ai_best_move()
1069
1070
1071     def do_sunfish_turn():
1072         """Synchronous wrapper around the frame-stepped search (used by tests and
1073         as a blocking fallback): runs the stepper until the move has been made."""
1074         _ai_reset_search()
1075         guard = 0
1076         while game_status == 'ai_thinking' and guard < 1000000:
1077             step_sunfish_search()
1078             guard += 1
1079
1080
1081     def do_ai_turn(color='b'):
1082         """Executes the AI turn based on Sunfish values and difficulty settings."""
1083         global game_status
1084         best_score = -float('inf') if color == 'w' else float('inf')
1085         best_moves = []
1086         sign = 1 if color == 'w' else -1
1087
1088         # Calculate noise level based on difficulty
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 18)

```
1089     noise_level = 1.0
1090     if difficulty == 0: noise_level = 200.0 # Easy
1091     elif difficulty == 1: noise_level = 25.0 # Medium
1092
1093     for r in range(8):
1094         for c in range(8):
1095             if (board[r][c] * sign) > 0:
1096                 moves = get_legal_moves(r, c)
1097                 for tr, tc in moves:
1098                     score = evaluate_move(r, c, tr, tc, color)
1099                     # Add random values to simulate mistakes based on difficulty
1100                     score += (py16.rnd(noise_level * 2) - noise_level)
1101
1102                 if color == 'w':
1103                     if score > best_score:
1104                         best_score = score
1105                         best_moves = [(r, c), (tr, tc)]
1106                     elif score == best_score:
1107                         best_moves.append((r, c), (tr, tc))
1108                 else:
1109                     if score < best_score:
1110                         best_score = score
1111                         best_moves = [(r, c), (tr, tc)]
1112                     elif score == best_score:
1113                         best_moves.append((r, c), (tr, tc))
1114
1115     if best_moves:
1116         import random
1117         move = random.choice(best_moves)
1118         execute_move(move[0][0], move[0][1], move[1][0], move[1][1])
1119     elif game_status == 'menu':
1120         # Fallback in demo mode if no more moves are possible
1121         reset_board(False)
1122         game_status = 'menu'
1123         return
1124
1125     if game_status == 'ai_thinking':
1126         game_status = 'play'
1127
1128 # --- CHESS RULE LOGIC ---
1129 def get_piece_color(val):
1130     if val > 0: return 'w'
1131     if val < 0: return 'b'
1132     return None
1133
1134 def get_pseudo_moves(r, c, bld):
1135     p = bld[r][c]
1136     if p == 0: return []
1137     color = 'w' if p > 0 else 'b'
1138     ptype = abs(p)
1139     moves = []
1140
1141     if ptype == 1:
1142         dr = -1 if color == 'w' else 1
1143         nr = r + dr
1144         if 0 <= nr < 8 and bld[nr][c] == 0:
1145             moves.append((nr, c))
1146             start_row = 6 if color == 'w' else 1
1147             if r == start_row and bld[r + 2*dr][c] == 0:
1148                 moves.append((r + 2*dr, c))
1149         for dc in [-1, 1]:
1150             nc = c + dc
1151             nr = r + dr
1152             if 0 <= nr < 8 and 0 <= nc < 8:
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 19)

```
1153         target = bld[nr][nc]
1154         if target != 0 and get_piece_color(target) != color:
1155             moves.append((nr, nc))
1156     if en_passant_target:
1157         tr, tc = en_passant_target
1158         if tr == r + dr and abs(tc - c) == 1:
1159             moves.append((tr, tc))
1160
1161     elif ptype == 2:
1162         offsets = [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]
1163         for dr, dc in offsets:
1164             nr, nc = r + dr, c + dc
1165             if 0 <= nr < 8 and 0 <= nc < 8:
1166                 if bld[nr][nc] == 0 or get_piece_color(bld[nr][nc]) != color:
1167                     moves.append((nr, nc))
1168
1169     elif ptype in [3, 4, 5]:
1170         dirs = []
1171         if ptype == 3 or ptype == 5:
1172             dirs += [(1,1), (1,-1), (-1,1), (-1,-1)]
1173         if ptype == 4 or ptype == 5:
1174             dirs += [(1,0), (-1,0), (0,1), (0,-1)]
1175
1176         for dr, dc in dirs:
1177             nr, nc = r + dr, c + dc
1178             while 0 <= nr < 8 and 0 <= nc < 8:
1179                 target = bld[nr][nc]
1180                 if target == 0:
1181                     moves.append((nr, nc))
1182                 else:
1183                     if get_piece_color(target) != color:
1184                         moves.append((nr, nc))
1185                     break
1186             nr += dr
1187             nc += dc
1188
1189     elif ptype == 6:
1190         for dr in [-1, 0, 1]:
1191             for dc in [-1, 0, 1]:
1192                 if dr == 0 and dc == 0: continue
1193                 nr, nc = r + dr, c + dc
1194                 if 0 <= nr < 8 and 0 <= nc < 8:
1195                     if bld[nr][nc] == 0 or get_piece_color(bld[nr][nc]) != color:
1196                         moves.append((nr, nc))
1197     return moves
1198
1199 def is_in_check(color, bld):
1200     k_pos = None
1201     target_king = 6 if color == 'w' else -6
1202     for r in range(8):
1203         for c in range(8):
1204             if bld[r][c] == target_king:
1205                 k_pos = (r, c)
1206             break
1207     if k_pos: break
1208     if not k_pos: return False
1209
1210     enemy_color = 'b' if color == 'w' else 'w'
1211     for r in range(8):
1212         for c in range(8):
1213             piece = bld[r][c]
1214             if piece != 0 and get_piece_color(piece) == enemy_color:
1215                 if abs(piece) == 1:
1216                     dr = -1 if enemy_color == 'w' else 1
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 20)

```
1217             if r + dr == k_pos[0] and abs(c - k_pos[1]) == 1:
1218                 return True
1219             else:
1220                 if k_pos in get_pseudo_moves(r, c, bld):
1221                     return True
1222         return False
1223
1224 def get_legal_moves(r, c):
1225     p = board[r][c]
1226     color = get_piece_color(p)
1227     pseudo = get_pseudo_moves(r, c, board)
1228     legal = []
1229
1230     for tr, tc in pseudo:
1231         saved_target = board[tr][tc]
1232
1233         is_ep = (abs(p) == 1 and en_passant_target == (tr, tc))
1234         if is_ep:
1235             saved_ep_piece = board[r][tc]
1236             board[r][tc] = 0
1237
1238             board[tr][tc] = p
1239             board[r][c] = 0
1240
1241             if not is_in_check(color, board):
1242                 legal.append((tr, tc))
1243
1244             board[r][c] = p
1245             board[tr][tc] = saved_target
1246             if is_ep:
1247                 board[r][tc] = saved_ep_piece
1248
1249     if abs(p) == 6 and not king_moved[color] and not is_in_check(color, board):
1250         if color == 'w' and r == 7 and c == 4:
1251             if not rook_moved['w'][1] and board[7][7] == 4 and board[7][5] == 0 and board[
1252                 board[7][5] = 6; board[7][4] = 0
1253                 if not is_in_check('w', board): legal.append((7, 6))
1254                 board[7][4] = 6; board[7][5] = 0
1255             if not rook_moved['w'][0] and board[7][0] == 4 and board[7][1] == 0 and board[
1256                 board[7][3] = 6; board[7][4] = 0
1257                 if not is_in_check('w', board): legal.append((7, 2))
1258                 board[7][4] = 6; board[7][3] = 0
1259         elif color == 'b' and r == 0 and c == 4:
1260             if not rook_moved['b'][1] and board[0][7] == -4 and board[0][5] == 0 and board
1261                 board[0][5] = -6; board[0][4] = 0
1262                 if not is_in_check('b', board): legal.append((0, 6))
1263                 board[0][4] = -6; board[0][5] = 0
1264             if not rook_moved['b'][0] and board[0][0] == -4 and board[0][1] == 0 and board
1265                 board[0][3] = -6; board[0][4] = 0
1266                 if not is_in_check('b', board): legal.append((0, 2))
1267                 board[0][4] = -6; board[0][3] = 0
1268
1269     return legal
1270
1271 def has_any_legal_moves(color):
1272     for r in range(8):
1273         for c in range(8):
1274             if board[r][c] != 0 and get_piece_color(board[r][c]) == color:
1275                 if len(get_legal_moves(r, c)) > 0:
1276                     return True
1277     return False
1278
1279 def position_key():
1280     """Compact FEN-like key of the current position for the repetition check."""
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 21)

```
1281     rows = "|".join(",".join(str(v) for v in board[r]) for r in range(8))
1282     castle = (
1283         int(king_moved['w']), int(rook_moved['w'][0]), int(rook_moved['w'][1]),
1284         int(king_moved['b']), int(rook_moved['b'][0]), int(rook_moved['b'][1])
1285     )
1286     ep = en_passant_target if en_passant_target else "-"
1287     return f"{rows};{turn};{castle};{ep}"
1288
1289
1290 def record_position():
1291     """Stores the current position key (used for threefold repetition)."""
1292     position_history.append(position_key())
1293
1294
1295 def is_insufficient_material():
1296     """True for dead-draw material: K-K, K+minor-K, same-coloured bishops only."""
1297     knights = 0
1298     bishops = 0
1299     bishop_sq_colors = set()
1300     for r in range(8):
1301         for c in range(8):
1302             p = board[r][c]
1303             if p == 0:
1304                 continue
1305             t = abs(p)
1306             if t == 6:
1307                 continue
1308             if t in (1, 4, 5):
1309                 return False # pawn, rook or queen -> mate possible
1310             if t == 2:
1311                 knights += 1
1312             elif t == 3:
1313                 bishops += 1
1314                 bishop_sq_colors.add((r + c) % 2)
1315     if knights == 0 and bishops == 0:
1316         return True # K vs K
1317     if knights + bishops == 1:
1318         return True # K+N vs K or K+B vs K
1319     if knights == 0 and len(bishop_sq_colors) == 1:
1320         return True # only bishops, all on one square colour
1321     return False
1322
1323
1324 def declare_draw(reason):
1325     global game_status, winner, draw_reason
1326     game_status = 'draw'
1327     winner = 'draw'
1328     draw_reason = reason
1329     play_sound(300, 400, py16.WAVE_TRIANGLE)
1330
1331
1332 def check_game_over_conditions():
1333     global game_status, winner
1334     if not has_any_legal_moves(turn):
1335         if is_in_check(turn, board):
1336             game_status = 'checkmate'
1337             winner = 'b' if turn == 'w' else 'w'
1338             play_sound(200, 600, py16.WAVE_SAW)
1339         else:
1340             declare_draw("STALEMATE")
1341     return
1342
1343     # Automatic draws while the game is still in progress
1344     if is_insufficient_material():
```



## PY16CHESS SUNFISH - CODE LISTING (PAGE 22)

```
1345         declare_draw("DRAW: MATERIAL")
1346     elif halfmove_clock >= 100:
1347         declare_draw("DRAW: 50-MOVE RULE")
1348     elif position_history and position_history.count(position_history[-1]) >= 3:
1349         declare_draw("DRAW: REPETITION")
1350
1351
1352 def finalize_turn(is_capture):
1353     global selected_sq, valid_moves, turn
1354     selected_sq = None
1355     valid_moves = []
1356
1357     if is_capture:
1358         play_sound(180, 100, py16.WAVE_SAW, decay_ms=40)
1359     else:
1360         play_sound(440, 50, py16.WAVE_SQUARE, decay_ms=20)
1361
1362     turn = 'b' if turn == 'w' else 'w'
1363     record_position()
1364     check_game_over_conditions()
1365
1366 def _snapshot_state():
1367     """A deep-ish copy of everything a move can change, for undo."""
1368     return {
1369         "board": [row[:] for row in board],
1370         "turn": turn,
1371         "king_moved": {"w": king_moved["w"], "b": king_moved["b"]},
1372         "rook_moved": {"w": rook_moved["w"][:], "b": rook_moved["b"][:]},
1373         "en_passant_target": en_passant_target,
1374         "halfmove_clock": halfmove_clock,
1375         "position_history": position_history[:],
1376         "game_status": game_status,
1377         "winner": winner,
1378         "draw_reason": draw_reason,
1379         "frames_white": frames_white,
1380         "frames_black": frames_black,
1381     }
1382
1383
1384 def push_undo():
1385     """Records the current state so the move about to be played can be undone."""
1386     undo_stack.append(_snapshot_state())
1387     if len(undo_stack) > MAX_UNDO:
1388         undo_stack.pop(0)
1389
1390
1391 def can_undo():
1392     return len(undo_stack) > 0
1393
1394
1395 def undo_move():
1396     """Reverts the last half-move (or, vs the AI, the last full move so the
1397     human keeps the move). Returns True if anything was undone."""
1398     global board, turn, king_moved, rook_moved, en_passant_target
1399     global halfmove_clock, position_history, game_status, winner, draw_reason
1400     global frames_white, frames_black, selected_sq, valid_moves, promotion_state
1401
1402     # Versus the AI, one tap should undo both the AI reply and your own move,
1403     # otherwise it would just be the AI's turn again.
1404     steps = 1
1405     if vs_ai and len(undo_stack) >= 2:
1406         steps = 2
1407
1408     s = None
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 23)

```
1409     for _ in range(steps):
1410         if not undo_stack:
1411             break
1412         s = undo_stack.pop()
1413
1414     if s is None:
1415         return False
1416
1417     board = s["board"]
1418     turn = s["turn"]
1419     king_moved = s["king_moved"]
1420     rook_moved = s["rook_moved"]
1421     en_passant_target = s["en_passant_target"]
1422     halfmove_clock = s["halfmove_clock"]
1423     position_history = s["position_history"]
1424     game_status = s["game_status"]
1425     winner = s["winner"]
1426     draw_reason = s["draw_reason"]
1427     frames_white = s["frames_white"]
1428     frames_black = s["frames_black"]
1429     selected_sq = None
1430     valid_moves = []
1431     promotion_state = None
1432     _ai_reset_search()
1433     return True
1434
1435
1436 # Standard material point values, indexed by abs(piece type).
1437 _MATERIAL_VALUES = {1: 1, 2: 3, 3: 3, 4: 5, 5: 9, 6: 0}
1438 # Full set each side starts with, for deriving what's been captured.
1439 _FULL_SET = {1: 8, 2: 2, 3: 2, 4: 2, 5: 1, 6: 1}
1440
1441
1442 def captured_by_color():
1443     """Returns (white_captured, black_captured) where each is a dict
1444     {piece_type: count} of ENEMY pieces that side has captured."""
1445     on_board_w = {t: 0 for t in _FULL_SET}
1446     on_board_b = {t: 0 for t in _FULL_SET}
1447     for r in range(8):
1448         for c in range(8):
1449             v = board[r][c]
1450             if v > 0:
1451                 on_board_w[v] += 1
1452             elif v < 0:
1453                 on_board_b[-v] += 1
1454     # White captured = black pieces missing from the board, and vice versa.
1455     white_captured = {t: _FULL_SET[t] - on_board_b[t] for t in _FULL_SET}
1456     black_captured = {t: _FULL_SET[t] - on_board_w[t] for t in _FULL_SET}
1457     return white_captured, black_captured
1458
1459
1460 def material_balance():
1461     """Positive = White is ahead by this many points, negative = Black ahead."""
1462     score = 0
1463     for r in range(8):
1464         for c in range(8):
1465             v = board[r][c]
1466             if v != 0:
1467                 pts = _MATERIAL_VALUES[abs(v)]
1468                 score += pts if v > 0 else -pts
1469     return score
1470
1471
1472 def find_king(color):
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 24)

```
1473     """(row, col) of the given side's king, or None."""
1474     target = 6 if color == 'w' else -6
1475     for r in range(8):
1476         for c in range(8):
1477             if board[r][c] == target:
1478                 return (r, c)
1479     return None
1480
1481
1482 def execute_move(sr, sc, r, c, ai_promo=None):
1483     """Central function for executing and documenting a move."""
1484     global was_capture_saved, en_passant_target, promotion_state, game_status, promote_cur
1485
1486     # Record undo state for every real game move (not the menu background demo).
1487     if game_status != 'menu':
1488         push_undo()
1489
1490     p = board[sr][sc]
1491     was_capture_saved = board[r][c] != 0 or (abs(p) == 1 and en_passant_target == (r, c))
1492
1493     # 50-move rule: reset on captures and pawn moves, otherwise count up
1494     if abs(p) == 1 or was_capture_saved:
1495         halfmove_clock = 0
1496     else:
1497         halfmove_clock += 1
1498
1499     if abs(p) == 1 and en_passant_target == (r, c):
1500         board[sr][c] = 0
1501
1502     if abs(p) == 6 and abs(sc - c) == 2:
1503         if c == 6:
1504             board[r][5] = board[r][7]
1505             board[r][7] = 0
1506         elif c == 2:
1507             board[r][3] = board[r][0]
1508             board[r][0] = 0
1509
1510     board[r][c] = board[sr][sc]
1511     board[sr][sc] = 0
1512
1513     if abs(p) == 6:
1514         king_moved[turn] = True
1515     if abs(p) == 4:
1516         if sr == 7:
1517             if sc == 0: rook_moved['w'][0] = True
1518             if sc == 7: rook_moved['w'][1] = True
1519         elif sr == 0:
1520             if sc == 0: rook_moved['b'][0] = True
1521             if sc == 7: rook_moved['b'][1] = True
1522
1523     next_ep_target = None
1524     if abs(p) == 1 and abs(sr - r) == 2:
1525         next_ep_target = ((sr + r) // 2, sc)
1526     en_passant_target = next_ep_target
1527
1528     if abs(p) == 1 and (r == 0 or r == 7):
1529         promotion_state = (r, c)
1530         promote_cursor = 0
1531         if game_status == 'menu' or game_status == 'ai_thinking' or (vs_ai and turn == 'b'):
1532             ptype = ai_promo if ai_promo else 5 # honour AI underpromotion, else queen
1533             board[r][c] = ptype if p > 0 else -ptype
1534             promotion_state = None
1535             finalize_turn(was_capture_saved)
1536     else:
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 25)

```
1537         game_status = 'promote'
1538     else:
1539         finalize_turn(was_capture_saved)
1540
1541
1542     # --- MAIN LOOP ---
1543     def update():
1544         global selected_sq, valid_moves, turn, game_status, board, sound_on, difficulty
1545         global cursor_r, cursor_c, promote_cursor, msg_timer, menu_cursor, settings_cursor, de
1546         global time_idx, frames_white, frames_black, bg_idx, winner
1547         global pending_move, confirm_moves
1548
1549         if msg_timer > 0:
1550             msg_timer -= 1
1551
1552         if game_status == 'menu':
1553             # In the menu we listen for any input (player=0)
1554             if py16.btnp('up', player=0):
1555                 menu_cursor = max(0, menu_cursor - 1)
1556                 play_sound(440, 30, py16.WAVE_TRIANGLE)
1557             if py16.btnp('down', player=0):
1558                 menu_cursor = min(4, menu_cursor + 1)
1559                 play_sound(440, 30, py16.WAVE_TRIANGLE)
1560
1561             # Hybrid mouse support in the menu
1562             if py16.mouse_btnp(0):
1563                 mx, my = py16.mouse_x(), py16.mouse_y()
1564                 if 40 <= mx <= 216:
1565                     if 90 <= my <= 110:
1566                         menu_cursor = 0
1567                         reset_board(True)
1568                         game_status = 'play'
1569                         play_sound(600, 50, py16.WAVE_SQUARE)
1570                         return
1571                     elif 110 <= my <= 130:
1572                         menu_cursor = 1
1573                         reset_board(False)
1574                         game_status = 'play'
1575                         play_sound(600, 50, py16.WAVE_SQUARE)
1576                         return
1577                     elif 130 <= my <= 150:
1578                         menu_cursor = 2
1579                         load_game()
1580                         play_sound(600, 50, py16.WAVE_SQUARE)
1581                         return
1582                     elif 150 <= my <= 170:
1583                         menu_cursor = 3
1584                         game_status = 'settings'
1585                         play_sound(600, 50, py16.WAVE_SQUARE)
1586                         return
1587                     elif 170 <= my <= 190:
1588                         menu_cursor = 4
1589                         game_status = 'license'
1590                         play_sound(600, 50, py16.WAVE_SQUARE)
1591                         return
1592
1593             # Confirm
1594             if py16.btnp('z', player=0) or py16.btnp('enter', player=0):
1595                 if menu_cursor == 0:
1596                     reset_board(True)
1597                     game_status = 'play'
1598                 elif menu_cursor == 1:
1599                     reset_board(False)
1600                     game_status = 'play'
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 26)

```
1601         elif menu_cursor == 2:
1602             load_game()
1603         elif menu_cursor == 3:
1604             game_status = 'settings'
1605         elif menu_cursor == 4:
1606             game_status = 'license'
1607         play_sound(600, 50, py16.WAVE_SQUARE)
1608         return
1609
1610     # --- DEMO AI ---
1611     demo_timer -= 1
1612     if demo_timer <= 0:
1613         demo_timer = 40 # One move every 40 frames
1614         do_ai_turn(turn)
1615         if game_status in ['checkmate', 'stalemate', 'draw']:
1616             reset_board(False)
1617             game_status = 'menu'
1618     return
1619
1620 if game_status == 'settings':
1621     if py16.btnp('up', player=0):
1622         settings_cursor = max(0, settings_cursor - 1)
1623         play_sound(440, 30, py16.WAVE_TRIANGLE)
1624     if py16.btnp('down', player=0):
1625         settings_cursor = min(5, settings_cursor + 1)
1626         play_sound(440, 30, py16.WAVE_TRIANGLE)
1627
1628     dir = 0
1629     if py16.btnp('left', player=0): dir = -1
1630     elif py16.btnp('right', player=0): dir = 1
1631
1632     action = py16.btnp('z', player=0) or py16.btnp('enter', player=0)
1633
1634     if py16.mouse_btnp(0):
1635         mx, my = py16.mouse_x(), py16.mouse_y()
1636         if 40 <= mx <= 216:
1637             if 90 <= my <= 108: settings_cursor = 0; dir = 1
1638             elif 108 <= my <= 126: settings_cursor = 1; dir = 1
1639             elif 126 <= my <= 144: settings_cursor = 2; dir = 1
1640             elif 144 <= my <= 162: settings_cursor = 3; dir = 1
1641             elif 162 <= my <= 180: settings_cursor = 4; dir = 1
1642             elif 180 <= my <= 198: settings_cursor = 5; action = True
1643
1644     # Toggle settings or press back button
1645     if dir != 0 or action:
1646         if dir == 0 and action: dir = 1 # Z-Key acts as a right arrow step
1647
1648         if settings_cursor == 0:
1649             difficulty = (difficulty + dir) % 3
1650             play_sound(440, 30, py16.WAVE_TRIANGLE)
1651         elif settings_cursor == 1:
1652             time_idx = (time_idx + dir) % 8
1653             play_sound(440, 30, py16.WAVE_TRIANGLE)
1654         elif settings_cursor == 2:
1655             sound_on = not sound_on
1656             play_sound(440, 30, py16.WAVE_TRIANGLE)
1657         elif settings_cursor == 3:
1658             bg_idx = (bg_idx + dir) % len(bg_options)
1659             play_sound(440, 30, py16.WAVE_TRIANGLE)
1660         elif settings_cursor == 4:
1661             confirm_moves = not confirm_moves
1662             play_sound(440, 30, py16.WAVE_TRIANGLE)
1663         elif settings_cursor == 5 and action:
1664             game_status = 'menu'
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 27)

```
1665         play_sound(600, 50, py16.WAVE_SQUARE)
1666     return
1667
1668     if game_status == 'license':
1669         if py16.btnp('z', player=0) or py16.btnp('space', player=0) or py16.btnp('enter',
1670             game_status = 'menu'
1671             play_sound(600, 50, py16.WAVE_SQUARE)
1672         return
1673
1674     # --- TIME CONTROL LOGIC ---
1675     if game_status == 'play' and time_idx > 0:
1676         if turn == 'w':
1677             frames_white += 1
1678             if frames_white >= time_options[time_idx] * 60:
1679                 game_status = 'checkmate'
1680                 winner = 'b'
1681                 play_sound(200, 600, py16.WAVE_SAW)
1682             elif turn == 'b' and not (vs_ai and game_status == 'ai_thinking'):
1683                 frames_black += 1
1684                 if frames_black >= time_options[time_idx] * 60:
1685                     game_status = 'checkmate'
1686                     winner = 'w'
1687                     play_sound(200, 600, py16.WAVE_SAW)
1688
1689     # Trigger for AI moves
1690     if game_status == 'play':
1691         if vs_ai and turn == 'b':
1692             game_status = 'ai_thinking'
1693             _ai_reset_search()      # start a fresh, frame-stepped search
1694             return # Renders the "AI THINKING..." label in the draw call
1695
1696     if game_status == 'ai_thinking':
1697         step_sunfish_search()      # one frame's slice; plays the move when ready
1698         return
1699
1700     # Saving and loading can be triggered by anyone (player=0)
1701     if py16.btnp('s', player=0):
1702         save_game()
1703         return
1704     if py16.btnp('a', player=0):
1705         load_game()
1706         return
1707
1708     # Undo the last move (X). Allowed during play or after a finished game.
1709     if py16.btnp('x', player=0):
1710         if game_status in ['play', 'checkmate', 'stalemate', 'draw'] and can_undo():
1711             if undo_move():
1712                 pending_move = None
1713                 play_sound(500, 60, py16.WAVE_TRIANGLE)
1714             else:
1715                 play_sound(200, 80, py16.WAVE_SAW)
1716         else:
1717             play_sound(200, 80, py16.WAVE_SAW)
1718         return
1719
1720     # Back to menu
1721     if game_status in ['checkmate', 'stalemate', 'draw']:
1722         if py16.btnp('space', player=0) or py16.btnp('enter', player=0):
1723             game_status = 'menu'
1724             return
1725     else:
1726         if py16.btnp('space', player=0):
1727             game_status = 'menu'
1728             return
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 28)

```
1729
1730 # Menu for pawn promotion (Uses the active controller)
1731 if game_status == 'promote':
1732     cp = get_active_controller()
1733     if py16.btnp('left', player=cp): promote_cursor = max(0, promote_cursor - 1)
1734     if py16.btnp('right', player=cp): promote_cursor = min(3, promote_cursor + 1)
1735
1736     action = False
1737     if py16.btnp('z', player=cp) or py16.btnp('enter', player=cp):
1738         action = True
1739     elif py16.mouse_btnp(0):
1740         mx = py16.mouse_x()
1741         my = py16.mouse_y()
1742         bx = BOARD_X
1743         by = BOARD_Y + 48
1744         if bx <= mx < bx + 128 and by <= my < by + 32:
1745             promote_cursor = (mx - bx) // 32
1746             action = True
1747
1748     if action:
1749         pr, pc = promotion_state
1750         p_color = 1 if pr == 0 else -1
1751         types = [5, 4, 3, 2]
1752         board[pr][pc] = types[promote_cursor] * p_color
1753
1754         game_status = 'play'
1755         promotion_state = None
1756         finalize_turn(was_capture_saved)
1757     return
1758
1759 if game_status != 'play':
1760     return
1761
1762 # Controller mapping for the current game
1763 cp = get_active_controller()
1764
1765 if py16.btnp('left', player=cp): cursor_c = max(0, cursor_c - 1); play_sound(800, 10,
1766 if py16.btnp('right', player=cp): cursor_c = min(7, cursor_c + 1); play_sound(800, 10,
1767 if py16.btnp('up', player=cp): cursor_r = max(0, cursor_r - 1); play_sound(800, 10, py
1768 if py16.btnp('down', player=cp): cursor_r = min(7, cursor_r + 1); play_sound(800, 10,
1769
1770 action = False
1771 r, c = cursor_r, cursor_c
1772
1773 # Action button
1774 if py16.btnp('z', player=cp) or py16.btnp('enter', player=cp):
1775     action = True
1776 elif py16.mouse_btnp(0):
1777     mx = py16.mouse_x()
1778     my = py16.mouse_y()
1779     if BOARD_X <= mx < BOARD_X + 128 and BOARD_Y <= my < BOARD_Y + 128:
1780         c = (mx - BOARD_X) // SQ_SIZE
1781         r = (my - BOARD_Y) // SQ_SIZE
1782         cursor_r, cursor_c = r, c
1783         action = True
1784
1785 if action:
1786     if selected_sq and (r, c) in valid_moves:
1787         sr, sc = selected_sq
1788         if confirm_moves and pending_move != (sr, sc, r, c):
1789             # First press selects the destination; show it and wait for a
1790             # second press on the same square to confirm.
1791             pending_move = (sr, sc, r, c)
1792             play_sound(700, 30, py16.WAVE_TRIANGLE)
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 29)

```
1793         else:
1794             pending_move = None
1795             execute_move(sr, sc, r, c)
1796     else:
1797         pending_move = None
1798         piece = board[r][c]
1799         if piece != 0 and get_piece_color(piece) == turn:
1800             selected_sq = (r, c)
1801             valid_moves = get_legal_moves(r, c)
1802             play_sound(600, 30, py16.WAVE_TRIANGLE)
1803         else:
1804             selected_sq = None
1805             valid_moves = []
1806
1807 def draw_piece(p, x, y):
1808     """Draws a chess piece from memory."""
1809     if p == 0: return
1810     ptype = abs(p)
1811     is_black = p < 0
1812     kx = (ptype - 1) * 2
1813     ky = 2 if is_black else 0
1814     spr_id = ky * 32 + kx
1815     py16.spr(spr_id, x, y, w=2, h=2)
1816
1817 def draw():
1818     py16.cls(bg_options[bg_idx])
1819
1820     # --- BACKGROUND & CHESSBOARD (ALWAYS DRAWN) ---
1821     py16.rect(BOARD_X - 2, BOARD_Y - 2, 132, 132, 0)
1822
1823     files = "ABCDEFGH"
1824     for i in range(8):
1825         py16.text(files[i], BOARD_X + i * 16 + 5, BOARD_Y - 10, 6)
1826         py16.text(str(8 - i), BOARD_X - 10, BOARD_Y + i * 16 + 5, 6)
1827
1828     for r in range(8):
1829         for c in range(8):
1830             lx = BOARD_X + c * SQ_SIZE
1831             ly = BOARD_Y + r * SQ_SIZE
1832
1833             sq_color = 6 if (r + c) % 2 == 0 else 4
1834             py16.rectfill(lx, ly, SQ_SIZE, SQ_SIZE, sq_color)
1835
1836             if selected_sq == (r, c) and game_status not in ['menu', 'settings']:
1837                 py16.rect(lx, ly, SQ_SIZE, SQ_SIZE, 10)
1838                 py16.rect(lx + 1, ly + 1, SQ_SIZE - 2, SQ_SIZE - 2, 10)
1839
1840             if (r, c) in valid_moves and game_status not in ['menu', 'settings']:
1841                 py16.circfill(lx + 8, ly + 8, 2, 11)
1842
1843             draw_piece(board[r][c], lx, ly)
1844
1845     # --- CHECK INDICATOR ---
1846     # Highlight the king of whichever side is currently in check.
1847     if game_status in ['play', 'promote', 'checkmate']:
1848         for kcolor in ('w', 'b'):
1849             if is_in_check(kcolor, board):
1850                 ksq = find_king(kcolor)
1851                 if ksq:
1852                     kr, kc = ksq
1853                     klx = BOARD_X + kc * SQ_SIZE
1854                     kly = BOARD_Y + kr * SQ_SIZE
1855                     # Pulsing red frame (colour 8) around the checked king.
1856                     if (py16.t() // 12) % 2 == 0:
```



## PY16CHESS SUNFISH - CODE LISTING (PAGE 30)

```
1857                 py16.rect(klx, kly, SQ_SIZE, SQ_SIZE, 8)
1858                 py16.rect(klx + 1, kly + 1, SQ_SIZE - 2, SQ_SIZE - 2, 8)
1859
1860             # --- DRAW CLOCKS ---
1861             if time_idx > 0:
1862                 # Draw white clock
1863                 rem_w = max(0, time_options[time_idx] - (frames_white // 60))
1864                 min_w, sec_w = rem_w // 60, rem_w % 60
1865                 col_w = 7 if turn == 'w' and game_status in ['play', 'promote'] else 5
1866                 py16.text(f"{min_w}:{sec_w:02d}", BOARD_X + 135, BOARD_Y + 110, col_w)
1867
1868                 # Draw black clock
1869                 rem_b = max(0, time_options[time_idx] - (frames_black // 60))
1870                 min_b, sec_b = rem_b // 60, rem_b % 60
1871                 col_b = 7 if turn == 'b' and game_status in ['play', 'promote', 'ai_thinking'] else 5
1872                 py16.text(f"{min_b}:{sec_b:02d}", BOARD_X + 135, BOARD_Y + 10, col_b)
1873
1874             # --- MENU OVERLAYS ---
1875             if game_status in ['menu', 'settings', 'license']:
1876                 # Slightly darken the screen
1877                 py16.blend_mode("alpha", alpha=160)
1878                 py16.rectfill(0, 0, 256, 224, 0)
1879                 py16.blend_mode("normal")
1880
1881                 if game_status != 'license':
1882                     py16.text("PY-16 CHESS", 90, 64, 7)
1883
1884                 if game_status == 'menu':
1885                     col_ai = 10 if menu_cursor == 0 else 6
1886                     col_vs = 10 if menu_cursor == 1 else 6
1887                     col_re = 10 if menu_cursor == 2 else 6
1888                     col_set = 10 if menu_cursor == 3 else 6
1889                     col_lic = 10 if menu_cursor == 4 else 6
1890
1891                     py16.text("NEW GAME (VS AI)", 58, 100, col_ai)
1892                     py16.text("NEW GAME (LOCAL)", 58, 120, col_vs)
1893                     py16.text("LOAD GAME", 58, 140, col_re)
1894                     py16.text("SETTINGS", 58, 160, col_set)
1895                     py16.text("LICENSE", 58, 180, col_lic)
1896
1897                     cy = 100 + menu_cursor * 20
1898                     py16.text(">", 46, cy, 10)
1899
1900                 elif game_status == 'settings':
1901                     col_dif = 10 if settings_cursor == 0 else 6
1902                     col_tim = 10 if settings_cursor == 1 else 6
1903                     col_snd = 10 if settings_cursor == 2 else 6
1904                     col_bg = 10 if settings_cursor == 3 else 6
1905                     col_cfm = 10 if settings_cursor == 4 else 6
1906                     col_bck = 10 if settings_cursor == 5 else 6
1907
1908                     diff_strs = ["EASY", "MEDIUM", "HARD"]
1909                     py16.text("AI STRENGTH: " + diff_strs[difficulty], 58, 99, col_dif)
1910
1911                     py16.text("TIME LIMIT: " + time_option_strs[time_idx], 58, 117, col_tim)
1912
1913                     snd_txt = "SOUND: ON" if sound_on else "SOUND: OFF"
1914                     py16.text(snd_txt, 58, 135, col_snd)
1915
1916                     py16.text("BACKGROUND: " + bg_names[bg_idx], 58, 153, col_bg)
1917
1918                     cfm_txt = "CONFIRM MOVES: ON" if confirm_moves else "CONFIRM MOVES: OFF"
1919                     py16.text(cfm_txt, 58, 171, col_cfm)
1920
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 31)

```
1921         py16.text("BACK", 58, 189, col_bck)
1922
1923         cy = 99 + settings_cursor * 18
1924         py16.text(">", 46, cy, 10)
1925
1926     elif game_status == 'license':
1927         py16.text("LICENSE INFO", 88, 40, 10)
1928
1929         py16.text("AI ENGINE:", 40, 70, 6)
1930         py16.text("SUNFISH (THOMAS AHLE)", 40, 85, 7)
1931
1932         py16.text("PUBLISHED UNDER:", 40, 115, 6)
1933         py16.text("GNU GENERAL PUBLIC LICENSE V3", 40, 130, 7)
1934
1935         py16.text("PRESS ANY KEY TO RETURN", 52, 190, 8)
1936
1937     if msg_timer > 0 and game_status != 'license':
1938         col = 8 if (py16.t() // 10) % 2 == 0 else 7
1939         py16.text(show_msg, 72, 200, col)
1940     return
1941
1942 # --- NORMAL GAME UI ---
1943 py16.text("PY-16 CHESS", 100, 12, 7)
1944
1945 if msg_timer > 0:
1946     col = 10 if (py16.t() // 10) % 2 == 0 else 7
1947     py16.text(show_msg, 92, 36, col)
1948
1949 if game_status == 'ai_thinking':
1950     py16.text("AI THINKING...", 84, 26, 8)
1951 elif game_status == 'play' or game_status == 'promote':
1952     p_str = f"(P{get_active_player_label()})"
1953     status_text = f"WHITE TO MOVE {p_str}" if turn == 'w' else f"BLACK TO MOVE {p_str}"
1954     py16.text(status_text, 76, 26, 10 if turn == 'w' else 0)
1955     if game_status == 'play' and is_in_check(turn, board):
1956         chk_col = 8 if (py16.t() // 12) % 2 == 0 else 7
1957         py16.text("CHECK!", 112, 36, chk_col)
1958 elif game_status == 'checkmate':
1959     win_text = "MATE! WHITE WINS" if winner == 'w' else "MATE! BLACK WINS"
1960     py16.text(win_text, 76, 26, 8)
1961     py16.text("ENTER FOR MENU", 88, 190, 6)
1962 elif game_status == 'draw':
1963     py16.text(draw_reason if draw_reason else "DRAW", 72, 26, 14)
1964     py16.text("ENTER FOR MENU", 88, 190, 6)
1965 elif game_status == 'stalemate':
1966     py16.text("STALEMATE! DRAW", 80, 26, 14)
1967     py16.text("ENTER FOR MENU", 88, 190, 6)
1968
1969 # --- CAPTURED PIECES & MATERIAL BALANCE ---
1970 if game_status in ['play', 'promote', 'checkmate', 'stalemate', 'draw']:
1971     white_capt, black_capt = captured_by_color()
1972     order = [5, 4, 3, 2, 1] # Q R B N P
1973     letters = {1: "P", 2: "N", 3: "B", 4: "R", 5: "Q"}
1974
1975     def capt_str(capt):
1976         parts = []
1977         for t in order:
1978             if capt[t] > 0:
1979                 parts.append(letters[t] * capt[t] if capt[t] <= 3 else f"{letters[t]}x")
1980         return "".join(parts)
1981
1982     # White's captures sit just below the board, black's just above it.
1983     wy = BOARD_Y + 128 + 2
1984     by = BOARD_Y - 20
```

## PY16CHESS SUNFISH - CODE LISTING (PAGE 32)

```
1985     py16.text(capt_str(white_capt), BOARD_X, wy, 7)
1986     py16.text(capt_str(black_capt), BOARD_X, by, 5)
1987
1988     bal = material_balance()
1989     if bal != 0:
1990         who = "W" if bal > 0 else "B"
1991         py16.text(f"{who}+{abs(bal)}", BOARD_X + 100, BOARD_Y + 128 + 2, 11)
1992
1993     py16.text("S:SAVE A:LOAD X:UNDO", BOARD_X - 4, BOARD_Y + 138, 6)
1994     py16.text("SPACE: TO MENU", BOARD_X + 24, BOARD_Y + 148, 6)
1995
1996     # Controller Cursor
1997     if game_status == 'play':
1998         cx = BOARD_X + cursor_c * SQ_SIZE
1999         cy = BOARD_Y + cursor_r * SQ_SIZE
2000         blink_col = 8 if (py16.t() // 15) % 2 == 0 else 9
2001         py16.rect(cx, cy, SQ_SIZE, SQ_SIZE, blink_col)
2002         py16.rect(cx - 1, cy - 1, SQ_SIZE + 2, SQ_SIZE + 2, blink_col)
2003
2004         # Pending-move confirmation marker (when CONFIRM MOVES is on).
2005         if confirm_moves and pending_move is not None:
2006             _, _, ptr, ptc = pending_move
2007             px = BOARD_X + ptc * SQ_SIZE
2008             py = BOARD_Y + ptr * SQ_SIZE
2009             mark_col = 11 if (py16.t() // 10) % 2 == 0 else 7
2010             py16.rect(px, py, SQ_SIZE, SQ_SIZE, mark_col)
2011             py16.rect(px + 1, py + 1, SQ_SIZE - 2, SQ_SIZE - 2, mark_col)
2012             py16.text("CONFIRM?", 108, 36, mark_col)
2013
2014         # Promotion Menu
2015         if game_status == 'promote':
2016             bx = BOARD_X
2017             by = BOARD_Y + 48
2018             py16.rectfill(bx, by, 128, 32, 5)
2019             py16.rect(bx, by, 128, 32, 7)
2020             py16.text("CHOOSE PIECE:", bx + 4, by - 10, 7)
2021
2022             pr, pc = promotion_state
2023             p_color = 1 if pr == 0 else -1
2024             types = [5, 4, 3, 2]
2025
2026             for i in range(4):
2027                 ox = bx + i * 32
2028                 py16.rect(ox, by, 32, 32, 7)
2029                 draw_piece(types[i] * p_color, ox + 8, by + 8)
2030
2031             px = bx + promote_cursor * 32
2032             blink_col = 8 if (py16.t() // 15) % 2 == 0 else 9
2033             py16.rect(px, by, 32, 32, blink_col)
2034             py16.rect(px - 1, by - 1, 34, 34, blink_col)
2035
2036     if __name__ == "__main__":
2037         py16.run(update, draw, init)
2038
```